

As a Senior Backend Architect, I have designed this system to prioritize **Consistency, Auditability, and Security**. Given the requirements, a **Microservices Architecture** is recommended to isolate sensitive domains (e.g., Ledger vs. AI Insights) and scale independently.

1. Architecture Overview

- **API Gateway:** Kong or AWS API Gateway for rate limiting, SSL termination, and request routing.
- **Communication:**
 - * **Synchronous:** gRPC for internal service-to-service communication (low latency).
 - * **Asynchronous:** Apache Kafka for event-driven workflows (e.g., Transaction -> Fraud Engine -> Notification).
- **Database:** PostgreSQL (Primary) for ACID compliance. Redis for caching session data and account balances.

2. Services

- **Identity Service:** Auth, MFA, RBAC, and Profile management.
- **Ledger Service:** Atomic financial transactions and balance tracking.
- **Loan Service:** Workflow engine for applications and approvals.
- **Fraud Engine:** Real-time analysis of transaction patterns.
- **Insight Service:** Background processing of user data for financial visualization.
- **Notification Service:** Email/SMS/Push notifications for alerts.

3. Modules

- **Transaction Module:** Implements the Double-Entry Bookkeeping pattern.
- **Auth Module:** Handles JWT issuance and MFA validation.
- **Workflow Module:** State machines for loan statuses (Pending -> Under Review -> Approved/Rejected).

4. Authentication & Authorization

- **Auth:** OAuth2 + OpenID Connect.
- **MFA:** Time-based OTP (TOTP) via Google Authenticator or SMS.

- **Authorization:** Policy-Based Access Control (PBAC) using **Open Policy Agent (OPA)** to evaluate complex rules (e.g., "Employee can only approve loans if credit score > X").

5. Business Logic

- **Atomic Transactions:** Use database-level `START TRANSACTION`` with `SELECT FOR UPDATE`` to lock records during balance transfers.
- **Idempotency:** Every transaction request must include an `idempotency_key`` header to prevent double-charging on network retries.

6. Background Jobs

- **Task Queue:** RabbitMQ or Redis (BullMQ).
- **Jobs:**
 - * `FraudWorker``: Consumes transactions for anomaly analysis.
 - * `InsightWorker``: Generates monthly budget summaries.
 - * `MaintenanceWorker``: Cleans up logs and enforces password rotation alerts.

7. File Storage

- **AWS S3 (Encrypted):** For storing loan application supporting documents (PDFs/Scans).
- **Retention:** Implement S3 Lifecycle policies to transition files to Glacier after 7 years for compliance.

8. Error Handling & Logging

- **Global Error Handler:** Standardized JSON error response (e.g., `ERR_INSUFFICIENT_FUNDS``, `ERR_MFA_REQUIRED``).
- **Centralized Logging:** ELK Stack (Elasticsearch, Logstash, Kibana) or Datadog for searchable audit trails.
- **Distributed Tracing:** OpenTelemetry for tracking requests across microservices.

9. Folder Structure (Node.js/TypeScript Example)

```
/src
  /config      # Env variables, DB configs
  /controllers # Request handlers
```

```
/services    # Business logic
/models      # DB entities
/middleware  # Auth, Validation, Rate Limiting
/events      # Kafka producers/consumers
/utils       # Currency formatting, Hash utilities
/tests       # Unit/Integration tests
```

10. Recommended Libraries

- **Framework:** NestJS (Node.js) or Go (Gin/Fiber) for performance.
- **ORM:** Prisma or TypeORM (with Strict Type Safety).
- **Validation:** `Joi` or `Zod` (input validation is critical for banking).
- **Security:** `helmet` (headers), `bcrypt` (hashing), `speakeasy` (MFA/TOTP).
- **Testing:** `Jest` and `Supertest`.
- **Monitoring:** `Prometheus` + `Grafana`.

Architectural Note: The "Golden Rule" for Banking

To ensure the **3-second transaction requirement**, the Ledger Service must never perform heavy operations (like triggering an AI insight update) within the main request thread. Always hand off secondary tasks to an event bus (Kafka) to ensure the client receives a 200 OK immediately after the record is persisted.